MicroFrontEnds

Principios
Monolito vs MFE
Casos de uso
Impacto

Module Federation
Native Federation
Single-SPA
Mono/Multi Repo

Principios

Idea: dividir una app grande en unidades más pequeñas, e independientes

Desarrollo independiente, despliegue autónomo

Separación de responsabilidades por dominio de negocio

Autonomía de equipo: de UI a BackEnd

Monolito vs MFE

Monolito: Todo el negocio agrupado: UI, rutas, estado. Más acoplamiento. Despliegue global. Más dificultad para escalar equipos.

MFE: Mini-apps, actualizadas de forma independiente. Reduce bloqueo entre equipos. Complejidad adicional en la consistencia de UX y coordinación entre equipos. Sobrecarga de despliegues.

Casos de Uso

De MicroServicios en Back A MFE en Front

Distintos subdominios:
Facturación, Att al cliente, RRHH

Migraciones parciales por cambios de versión

Multinacionales: cambios regulatorios o imagen en distintos mercados

Impacto

Positivo: Despliegue independiente. Equipos autónomos. Desacoplamiento del código. Flexibilidad tecnológica. Mejoras de rendimiento (lazy loading).

Negativo: Mayor complejidad. Gestión dependencias. Tamaño de Host. Gestión del estado. Complicación CI.

Module Federation

Depende de WebPack. Angular < 16

Genera módulos propios (ni ESM ni UMD)

Shell (o Host) + Remote (otras apps)

Compartición de dependencias vía Runtime Webpack

Module Federation

A usar cuando varias aplicaciones comparten el mismo framework y necesitan integrarse compartiendo dependencias en tiempo de ejecución.

Native Federation

Genera módulos estándar ESM

El navegador resuelve módulos

Shell (o Host) + Remote (otras apps)

Compartición de dependencias mediante imports

Native Federation

A usar cuando s se quiere una federación similar a Module Federation pero basada en estándares del navegador (ESM + import maps) y no dependiente de Webpack.

Single SPA

Micros heterogéneos (React, Angular, etc.)

Comunicación vía Eventos / Api Navegador

Shell (o Host) + Apps independientes

Single-SPA orquesta la carga de apps

Single SPA

A usar cuando se quiere orquestar aplicaciones completamente independientes, incluso con frameworks distintos, manteniendo un acoplamiento mínimo entre ellas.

Mono/Multi Repo

Mono: Fácil compartir código. Cambios versiones coordinados. Herramientas comunes (build, tests). Repo grande. Mayor coordinación entre equipos.

Multi: Equipos independientes. Despliegues desacoplados. Mayor complejidad en la gestión. Compartir código es más difícil.